

C 语言基础常见问题讲解

C1 上机赛后基础知识整理

怎样使用这份讲义

C 语言是一种用来编写计算机程序的语言。**程序**是一组按顺序交给计算机执行的指令。我们写下的程序文字叫作**源代码**，平时也简称为**代码**。

这份讲义从一个完整程序开始，依次解释输入输出、条件判断、分支、循环、比较、求和和程序框架。每遇到一个新名词，文中会先说明它的含义，再使用它。阅读示例时，请试着用自己的话说出每一行在做什么。只记住代码的形状，却说不出它的作用，换一道题时仍然容易出错。

文中有两种示例。**C 代码片段**按照 C 语言的书写规则来写；这些书写规则统称为**语法**。代码片段有时需要放进完整程序，并先准备好其中使用的变量，才能运行。标注“可直接编译运行”的示例是完整程序。把 C 源代码翻译成计算机可以执行的程序，这个过程叫作**编译**，完成翻译的工具叫作**编译器**。

伪代码是用接近代码的形式写出的解题步骤。它可以使用中文变量名，目的是先把思路说清楚；伪代码不能直接提交，也不能直接交给编译器运行。本文只在需要概括解题步骤时使用伪代码，并省略不影响思路的语法细节。

1 先认识一个完整的 C 程序

可直接编译运行的完整 C 程序

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello, C!\n");
6     return 0;
7 }
```

先逐行理解这个程序。

1. `#include <stdio.h>` 表示使用标准输入输出头文件。**头文件**提供某些工具的声明；这里的 `stdio.h` 让程序可以使用 `scanf` 和 `printf`。

2. `main` 是**主函数**。**函数**是一段完成特定工作的代码，主函数是程序开始执行的地方。
3. 一对花括号 `{ }` 圈出一个**代码块**，也就是属于同一结构的一组语句。
4. `printf("Hello, C!\n");` 是一条**语句**。语句是让程序完成一个动作的完整指令，末尾通常要写分号。
5. `return 0;` 表示主函数正常结束，程序执行完毕。

花括号表示范围，分号表示一条语句结束。它们的作用不同，不能互相代替。

2 scanf 和 printf 的具体用法

`scanf` 和 `printf` 都是函数。`scanf` 从键盘或题目给定的输入中读取数据，`printf` 把数据输出到屏幕或**评测系统**。评测系统会自动运行提交的程序，并检查输出是否与正确答案一致。

2.1 变量数据类型和格式串

变量是程序中用来保存数据的一块空间。变量有名字，也有**数据类型**。数据类型说明这块空间准备保存什么样的数据，例如整数、小数或字符。

```
1 int n;  
2 scanf("%d", &n);  
3 printf("%d\n", n);
```

第一行声明了变量 `n`。**声明变量**就是告诉程序变量的名字和数据类型。这里的 `int` 表示整数类型，因此 `n` 用来保存一个整数。

函数名后圆括号中交给函数使用的内容叫作**参数**。`scanf` 和 `printf` 的第一个参数放在双引号中，叫作**格式串**。格式串说明数据要按什么形式输入或输出。格式串中的 `%d` 叫作**格式说明符**，它为一个整数预留位置。

若输入是 18，第二行把 18 保存到 `n` 中。第三行在 `%d` 的位置输出 `n` 保存的整数，最后用 `\n` 换行。

2.2 常用的数据类型和格式说明符

数据类型	变量声明示例	scanf 中使用	printf 中使用
int	int n;	%d	%d
long long	long long s;	%lld	%lld
double	double x;	%lf	%f
char	char c;	%c	%c

int 保存整数。long long 也保存整数，但通常能保存比 int 更大的整数。double 保存带小数部分的数。char 保存一个字符，例如字母 A 或符号 +。

格式说明符必须与变量类型对应。类型不对应时，程序可能读错数据或输出错误结果。特别注意：double 在 scanf 中使用 %lf，在 printf 中使用 %f。

2.3 scanf 中为什么通常要写取地址符

程序运行时，变量暂时存放在计算机的内存中。scanf 要把读到的数据放进变量，所以必须知道变量在内存中的位置。变量名前的 & 叫作取地址符，表示“这个变量的位置”。

正确写法：

```
1 int n;  
2 scanf("%d", &n);
```

容易出错的写法：

```
1 int n;  
2 scanf("%d", n);
```

第二段代码没有把变量 n 的位置交给 scanf。程序可能崩溃，也可能得到无法预测的结果。printf 只读取变量中已有的值，不会向变量里存入新数据，因此输出普通变量时通常不写 &。

2.4 一次输入多个数据

格式串中有几个格式说明符，后面就要按相同顺序写几个变量。

```
1 int a, b;  
2 scanf("%d%d", &a, &b);  
3 printf("%d %d\n", a, b);
```

输入 3 8 后，变量 a 保存 3，变量 b 保存 8。读取整数时，scanf 会自动跳过整数前

面的空格和换行，所以 "%d%d" 和 "%d %d" 都能读取这两个整数。

格式串中的普通字符必须与输入对应。例如：

```
1 int y, m, d;
2 scanf("%d-%d-%d", &y, &m, &d);
```

这段代码要求输入类似 2026-9-16，因为格式串中写了两个减号。若题目只说“输入三个整数”，通常不要自行在格式串中加入逗号、冒号或减号。

2.5 转义字符和百分号

在双引号中，反斜杠与后面的字符可以合起来表示一个特殊动作。这样的组合叫作**转义字符**。

写法	含义	屏幕上看到的结果
\n	换行	后面的内容从下一行开始
\\	输出一个反斜杠	\
%%	在 printf 中输出百分号	%

```
1 printf("first line\nsecond line\n");
2 printf("C:\\code\\main.c\n");
3 printf("100%%\n");
```

第一条语句输出两行文字。第二条语句中每两个反斜杠在屏幕上显示成一个反斜杠。第三条语句用两个百分号输出一个百分号，因为单独的 % 会被当作格式说明符的开头。

2.6 输入输出出错时怎样检查

1. 变量是否已经声明，数据类型是否正确。
2. 格式说明符是否与变量类型对应。
3. scanf 中需要接收数据的普通变量前是否写了 &。
4. 格式串中是否误加了逗号、冒号或其他普通字符。
5. 格式说明符的数量是否与后面的变量数量相同，顺序是否一致。
6. 输出内容是否与题目要求完全相同，有没有多写说明文字或空格。

3 逻辑运算符和条件表达式

表达式是由数据、变量和运算符组成并能计算出一个结果的代码，例如 $a + b$ 。**运算符**是表示某种运算的符号，例如加号 $+$ 和大于号 $>$ 。

分支和循环需要判断某件事是否成立。用来进行这种判断的表达式叫作**条件表达式**。在 C 语言中，数值 0 表示条件不成立，也就是“假”；非 0 的值表示条件成立，也就是“真”。比较结果成立时通常得到 1，不成立时得到 0。

3.1 四个常见逻辑运算符

运算符	含义	条件示例
<code>==</code>	等于	<code>a == b</code> 判断两个值是否相等
<code>!=</code>	不等于	<code>a != b</code> 判断两个值是否不同
<code>&&</code>	并且	<code>a > 0 && b > 0</code> 要求两边都成立
<code> </code>	或者	<code>a == 0 b == 0</code> 只要求一边成立

要判断 x 是否在 1 到 10 之间，包括两个端点，应该把它拆成“ x 不小于 1”和“ x 不大于 10”两个条件：

```
1 if (x >= 1 && x <= 10)
2 {
3     printf("YES\n");
4 }
```

不能写成 `1 <= x <= 10`。C 会先计算 `1 <= x`，结果只能是 0 或 1，再拿这个结果与 10 比较。这与“ x 在区间内”的含义不同。

3.2 赋值和判断相等不是一回事

一个等号 `=` 是**赋值运算符**，作用是把右边的结果保存到左边的变量中。两个等号 `==` 是判断相等，只能用在需要判断的表达式中。

下面的完整程序先把 5 保存到变量 x ，再判断 x 中的值是否等于 5。

可直接编译运行的完整 C 程序

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int x;
```

```

6
7     x = 5;
8     if (x == 5)
9     {
10        printf("x 中保存的是 5\n");
11    }
12
13    return 0;
14 }

```

语句 `x = 5;` 把 5 保存到变量中。`if` 圆括号里的 `x == 5` 才是在提问：“变量中保存的值是否等于 5？”

如果在 `if` 的条件中误写一个等号，例如 `if (x = 5)`，程序会先把 5 存进 `x`。这个赋值表达式的结果是非 0，所以条件会被当成真。**编译警告**是编译器发现可疑写法后给出的提示；代码可能仍能运行，但通常需要检查。编译器往往会为这种写法给出警告，不要直接忽略。

3.3 并且和或者怎样判断

`&&` 表示“并且”，左右两边都成立，整个条件才成立。`||` 表示“或者”，左右两边只要有一边成立，整个条件就成立。

条件 A	条件 B	A && B	A B
假	假	假	假
假	真	假	真
真	假	假	真
真	真	真	真

条件较长时，先把题目的句子拆开，再用括号写清组合关系。例如，“年份能被 400 整除，或者能被 4 整除但不能被 100 整除”写成：

```

1  if (y % 400 == 0 ||
2      (y % 4 == 0 && y % 100 != 0))
3  {
4      printf("YES\n");
5  }

```

这里用变量 `y` 保存年份。`%` 是**取余运算符**，计算除法的余数。`y % 4 == 0` 表示年份除以 4 的余数是 0，也就是能被 4 整除。

4 正确书写分支结构

分支结构让程序根据条件选择不同的代码执行。`if` 表示“如果条件成立”，`else` 表示“否则”。`if` 和 `else` 是 C 语言预先规定好用途的词，叫作**关键字**，不能拿来当变量名。

4.1 `if` 和 `else` 的基本结构

下面是一个 C 代码片段。变量 `s` 保存分数；把它放进完整程序时，需要先给 `s` 输入一个值。

C 代码片段 需要放进完整程序，并准备好其中使用的变量

```
1 if (s >= 60)
2 {
3     printf("通过\n");
4 }
5 else
6 {
7     printf("未通过\n");
8 }
```

程序先检查圆括号里的条件。条件成立时，只执行第一个花括号中的代码；条件不成立时，只执行 `else` 后花括号中的代码。两个分支不会同时执行。

4.2 为什么建议始终写花括号

不写花括号时，`if` 只控制紧跟在后面的一条语句。缩进不会改变程序结构。

容易出错的写法：

```
1 if (x > 0)
2     printf("这是正数\n");
3     printf("检查结束\n");
```

看起来两条输出都向右缩进了，但实际上只有第一条属于 `if`，第二条每次都会执行。C 编译器根据花括号和分号理解结构，不会根据空格猜测我们的意思。

正确写法：

```
1 if (x > 0)
2 {
3     printf("这是正数\n");
4     printf("检查结束\n");
5 }
```

初学阶段，无论代码块中有几条语句，都写花括号。缩进帮助人看清层次，花括号把代码的实际范围告诉编译器。

4.3 多个互斥情况怎样写

互斥表示这些情况不能同时成立。例如，一个分数只能属于一个等级。程序从上到下检查同一组 `if`、`else if`、`else`；找到第一个成立的条件后，执行对应代码块，并跳过后面的分支。

下面的 C 代码片段约定变量 `s` 已经保存分数。

C 代码片段 需要放进完整程序，并准备好其中使用的变量

```
1  if (s >= 90)
2  {
3      printf("A\n");
4  }
5  else if (s >= 80)
6  {
7      printf("B\n");
8  }
9  else if (s >= 60)
10 {
11     printf("C\n");
12 }
13 else
14 {
15     printf("D\n");
16 }
```

如果程序检查到第二个条件，说明第一个条件已经不成立，所以此时分数一定小于 90。第二个条件只需写 `s >= 80`，不必重复写“并且分数小于 90”。

4.4 分支结构检查表

- 条件外有圆括号，分支代码外有花括号。
- 判断相等使用 `==`，不等使用 `!=`。
- `else` 后面不写条件；还要判断条件时写 `else if`。
- 不要在 `if`（条件）后误写分号。分号会形成一条什么也不做的**空语句**，使后面的代码块不再受这个 `if` 正常控制。
- 花括号成对出现，同一层代码保持相同缩进。

5 循环的写法和理解

循环结构让同一段代码重复执行。写循环前先回答两个问题：一共重复多少次，每次重复要做什么。

5.1 for 循环

当重复次数在循环开始前能够确定时，通常使用 `for`。

```
1 for (i = 0; i < n; i++)
2 {
3     printf("=");
4 }
```

变量 `i` 用来记录循环进行到了第几次，叫作**循环变量**。`for` 圆括号中的三部分用分号隔开：

1. `i = 0` 在循环开始前执行一次，把 `i` 的初值设为 0。给变量设置开始值叫作**初始化**。
2. `i < n` 在每轮开始前判断。花括号中每轮重复执行的代码叫作**循环体**。条件成立时执行循环体，条件不成立时结束循环。
3. `i++` 在每轮结束后执行一次。`i++` 表示让 `i` 增加 1，这一步叫作**更新循环变量**。

当 `n = 4` 时，`i` 依次为 0, 1, 2, 3，循环正好执行四次。`i = 4` 时，条件 `i < n` 不成立，循环结束。

5.2 while 循环

当我们更关心“满足什么条件时继续”，或者重复次数不能提前确定时，可以使用 `while`。

C 代码片段 需要放进完整程序，并准备好其中使用的变量

```
1 int i = 0;
2
3 while (i < n)
4 {
5     printf("=");
6     i++;
7 }
```

这段 C 代码片段与上一节的 `for` 循环作用相同。写 `while` 时必须注意三个位置：循环前设置初值，圆括号中写继续循环的条件，循环体内更新会影响条件的变量。若漏掉最后的

更新，条件可能永远成立，程序就不会结束。

5.3 在循环中输入变量是什么意思

假设要输入 n 个整数，并把它们相加。只需要一个变量保存“本轮刚输入的数”。下一轮输入时，新数会覆盖旧数，但旧数已经加进总和，所以不会影响结果。

伪代码 只说明步骤，不能直接交给编译器运行

```
1 总和 = 0
2 for (重复 n 次)
3 {
4     输入一个数
5     把这个数加到总和
6 }
7 输出总和
```

这叫作**边读边处理**：读到一个数，就立即完成与它有关的计算。如果后面还要再次使用每一个输入值，才需要把所有值保存下来。保存一组相同类型数据的常用结构叫作**数组**，数组会在后续课程中专门学习。

5.4 循环边界为什么容易差一

循环边界是循环变量能够取到的第一个值和最后一个值。下面两种写法都执行 n 次，但循环变量的取值不同：

```
1 for (i = 0; i < n; i++)
2 {
3 }
4
5 for (i = 1; i <= n; i++)
6 {
7 }
```

第一种写法中， i 从 0 到 $n - 1$ 。第二种写法中， i 从 1 到 n 。若从 0 开始却写成 $i \leq n$ ，就会多执行一次。检查时不要只看符号，要写出第一个值、最后一个值和总个数。

6 比大小求和和求最大值

6.1 比较两个数

下面给出一个可直接编译运行的完整程序。变量 `a` 和 `b` 保存输入的两个整数，变量 `m` 保存较大值。

可直接编译运行的完整 C 程序

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int a, b, m;
6
7      scanf("%d%d", &a, &b);
8
9      if (a > b)
10     {
11         m = a;
12     }
13     else
14     {
15         m = b;
16     }
17
18     printf("%d\n", m);
19     return 0;
20 }
```

如果两个数相等，条件 `a > b` 不成立，程序进入 `else`，把 `b` 作为较大值。因为两者相等，结果仍然正确。只有题目要求单独处理“相等”时，才需要增加一个分支。

6.2 求多个数的和

保存累加结果的变量叫作**累加变量**。求和前必须把它初始化为 0，因为从 0 开始加不会改变正确结果。

下面直接写成 C 代码片段。这里约定：`n` 表示数字个数，`i` 表示已经进行的循环次数，`x` 保存本轮输入，`s` 保存总和。

C 代码片段 需要放进完整程序，并准备好其中使用的变量

```
1  int n, i, x;
2  long long s = 0;
3
4  scanf("%d", &n);
```

```
5 for (i = 0; i < n; i++)
6 {
7     scanf("%d", &x);
8     s = s + x;
9 }
10
11 printf("%lld\n", s);
```

这里使用 long long 保存总和，是因为许多整数相加后，结果可能超过 int 通常能够保存的范围。

不要只写 long long s; 就开始累加。在函数内部声明但没有设置初值的变量，初始内容是不确定的。用不确定的值继续相加，结果也不确定。

6.3 求多个数的最大值

最稳妥的方法是先读入第一个数，把它当作“目前见过的最大值”，再逐个处理其余数字。

伪代码 只说明步骤，不能直接交给编译器运行

```
1 输入第一个数，作为当前最大值
2 for (依次处理其余数字)
3 {
4     输入一个数
5     if (这个数更大) 更新当前最大值
6 }
7 输出当前最大值
```

第一个数已经在循环前读入，所以循环只需再读 数字个数 - 1 个数，循环变量从 1 开始。这个方法要求数字个数至少为 1。

不要把最大值随手初始化为 0。若输入是 -8 -3 -10，所有输入都小于 0，程序就会错误地把没有出现在输入中的 0 当作最大值。用第一个输入值初始化，正数、零和负数都能正确处理。

求最小值时使用相同方法：先把第一个输入当作当前最小值，后面遇到更小的数时再更新。

7 写代码前先构建程序框架

题目示例：输入一个整数 n ，输出一个左尖括号、 n 个等号和一个右尖括号。若输入 5，输出：

<=====>

7.1 先把题目改写成明确步骤

输入只有整数 n 。输出由三部分组成：先输出 <，再重复输出 n 个 =，最后输出 >。只有中间部分重复，因此循环只包住输出等号的语句。

伪代码 只说明步骤，不能直接交给编译器运行

```
1 输入 n
2 输出 "<"
3 for (重复 n 次) 输出 "="
4 输出 ">" 并换行
```

7.2 再翻译成完整的 C 程序

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int n, i;
6
7     scanf("%d", &n);
8
9     printf("<");
10    for (i = 0; i < n; i++)
11    {
12        printf("=");
13    }
14    printf(">\n");
15
16    return 0;
17 }
```

变量 n 保存题目给出的等号数量，变量 i 控制循环。左尖括号和右尖括号各输出一次，所以放在循环外；等号要输出 n 次，所以放在循环内。

7.3 用小样例手动执行

手动执行是指不用计算机，自己按代码顺序记录变量和输出怎样变化。设 $n = 3$ ：

1. 先输出 $<$ 。
2. $i = 0$ ，条件 $0 < 3$ 成立，输出第一个 $=$ 。
3. 更新后 $i = 1$ ，条件成立，输出第二个 $=$ 。
4. 更新后 $i = 2$ ，条件成立，输出第三个 $=$ 。
5. 更新后 $i = 3$ ，条件 $3 < 3$ 不成立，循环结束。
6. 最后输出 $>$ 和换行，完整结果是 $<===>$ 。

还应测试 $n = 0$ 。此时循环一次也不执行，只输出 $<>$ 。这种刚好位于取值范围开头或结尾的数据叫作**边界情况**，它很适合检查循环次数是否正确。

8 常见错误的定位方法

8.1 编译错误

编译错误表示代码不符合 C 语言语法，编译器无法完成翻译。

先处理编译器报告的第一处错误，因为后面的错误可能只是被第一处错误连带引起。常见原因包括：

- 漏写分号。
- 圆括号、花括号或双引号没有成对。
- 变量没有声明，或者同一个变量名前后拼写不一致。
- 使用了中文全角符号，例如中文分号、中文引号或全角感叹号。

8.2 程序能运行但答案错误

程序通过编译并不表示思路一定正确。若程序能结束，但输出与题目答案不同，按下面的顺序检查：

1. 选一组很小的输入，自己先算出正确答案。
2. 检查是否读入了题目给出的全部数据，格式说明符是否正确。
3. 检查条件中的 $=$ 、 $==$ 、 $!=$ 、 $\&\&$ 和 $||$ 。

4. 写出循环变量从第一个值到最后一个值的完整取值，确认执行次数。
5. 检查求和变量是否从 0 开始，最大值或最小值是否用输入值初始化。
6. 检查输出有没有多余文字、空格或换行。

8.3 程序一直不结束

程序开始运行后长时间没有结束，常见原因是循环条件一直为真。检查循环体内是否改变了会影响条件的变量，以及改变方向是否能让条件最终变成假。

下面的 C 代码片段会一直循环，因为变量 `i` 没有在循环体中改变。

C 代码片段 需要放进完整程序，并准备好其中使用的变量

```
1 int i = 0;
2
3 while (i < 10)
4 {
5     printf("%d\n", i);
6 }
```

这里的 `i` 永远是 0，所以 `i < 10` 永远成立。要让循环结束，循环体内必须加入 `i++`。当 `i` 变成 10 时，条件为假，循环结束。

9 上机前快速检查表

提交代码前，从上到下检查一遍：

1. 是否包含需要的头文件和完整的 `main` 函数。
2. 每个变量是否已经声明，数据类型是否适合保存结果。
3. 变量是否在第一次使用前完成初始化或输入。
4. `scanf` 的格式说明符、取地址符和变量是否一一对应。
5. `printf` 是否只输出题目要求的内容。
6. 判断相等是否使用 `==`，不等是否使用 `!=`。
7. `if`、`else`、`for` 和 `while` 的代码块是否用花括号圈好。
8. 循环的初值、条件和更新是否能得到正确次数，并且最终能够结束。
9. 求和是否从 0 开始，最大值和最小值是否用输入值初始化。

10. 是否测试了边界情况，例如 $n = 0$ 、 $n = 1$ 、负数和相等的数。
11. 编译器是否仍有警告。警告表示代码中可能存在错误，不应直接忽略。

最后的练习方法

练习时不要求一次写对。请形成固定顺序：确定输入和输出，写出步骤，选择分支或循环，初始化变量，再用边界情况检查。读懂示例后，合上讲义重新写代码，并用一组很小的数据手动执行。若结果不对，就定位到具体一行，依次说明它要做什么、执行前变量保存什么值、执行后变量发生什么变化。每一步都能用自己的话解释清楚时，代码便不再是需要死记的符号。